MLOPS ASSIGNMENT REPORT

END-TO-END ML MODEL DEVELOPMENT, CI/CD, AND PRODUCTION DEPLOYMENT

HEART DISEASE PREDICTION USING MACHINE LEARNING
COURSE: MLOPS (S2-25_AMLCSZG523)

GITHUB REPOSITORY:
HTTPS://GITHUB.COM/TUSHARCHOUHAN/HEART-DISEASE-MLOPS

DEPLOOYMENT LINK:
https://heart-disease-mlops-951222900001.us-central1.run.app/

VIDEO RECORDING:
HTTPS://GITHUB.COM/TUSHARCHOUHAN/HEART-DISEASE-MLOPS/TREE/MAIN/DEMO%20VIDEO

Tusar Chouhan (2025CS05043)

DATE: 4 MAY 2026

Table of Contents

1. Introduction 2

2. Setup & Installation Instructions 3

3. Data Acquisition & Exploratory Data Analysis 4

4. Feature Engineering & Model Development 6

5. Experiment Tracking with MLflow 8

6. Model Packaging & Reproducibility 10

7. CI/CD Pipeline & Automated Testing 11

8. Model Containerization 12

9. Production Deployment 13

10. Monitoring & Logging 14

11. Architecture Diagram 16

12. Conclusion & Future Work 17

13. References 18

1. Introduction

1.1 Problem Statement

Heart disease is one of the leading causes of death globally. Early detection and risk assessment can significantly improve patient outcomes. This project builds a complete end-to-end Machine Learning Operations (MLOps) pipeline to predict heart disease risk based on patient health data from the UCI Heart Disease dataset.

The solution encompasses the full ML lifecycle: data acquisition, exploratory analysis, model development, experiment tracking, automated testing, containerization, cloud deployment, and production monitoring — following industry-standard MLOps best practices.

1.2 Dataset Description

- The Heart Disease UCI dataset contains 303 patient records with 14 features:
    - Demographic: age, sex
    - Clinical: chest pain type (cp), resting blood pressure (trestbps), cholesterol (chol),
    - fasting blood sugar (fbs), resting ECG (restecg)
    - Exercise: maximum heart rate (thalach), exercise-induced angina (exang),
    - ST depression (oldpeak), slope of ST segment (slope)
    - Diagnostic: number of major vessels (ca), thalassemia (thal)
    - Target: presence (1) or absence (0) of heart disease

- Data Source: UCI Machine Learning Repository

1.3 Technology Stack

Component	Technology
Programming Language	Python 3.11
ML Framework	Scikit-learn 1.5.2
API Framework	FastAPI 0.115.6
Experiment Tracking	MLflow 2.19.0
Testing	Pytest 8.3.4
CI/CD	GitHub Actions
Containerization	Docker
Cloud Deployment	Google Cloud Run
Monitoring	Prometheus Client + Logging

2. Setup & Installation Instructions

2.1 Prerequisites

- Python 3.11+
- Git
- Google Cloud SDK (for deployment)

2.2 Local Setup

Step	Command / Action
Step 1: Clone the repository	`git clone https://github.com/tusharchouhan/heart-disease-mlops.gitcd heart-disease-mlops`
Step 2: Create and activate virtual environment	`python -m venv venvvenv\Scripts\activate` (Windows)`source venv/bin/activate` (Linux/Mac)
Step 3: Install dependencies	`pip install -r requirements.txtpip install -e .`
Step 4: Download dataset	`python src/data/download_data.py`
Step 5: Run EDA	`python notebooks/heart-disease-data-eda.ipynb`
Step 6: Train models with experiment tracking	`python src/models/train_mlflow.py`
Step 7: Run unit tests	`python -m pytest tests/ -v`
Step 8: Start API server	`python -m uvicorn src.api.app:app --host 0.0.0.0 --port 8000`
Step 9: View MLflow dashboard	`mlflow ui --backend-store-uri sqlite:///mlflow.db`

3. Data Acquisition & Exploratory Data Analysis

3.1 Data Acquisition

The dataset was downloaded programmatically from the UCI Machine Learning Repository using a custom download script (src/data/download_data.py). The script:

- Downloads raw data from the UCI repository URL
- Assigns proper column names to all 14 features
- Convert the multi-class target variable to binary (0/1)
- Saves the cleaned CSV to the data/ directory

3.2 Data Cleaning

Missing values were identified in the 'ca' and 'thal' columns (represented as '?' in the original dataset). These were handled by imputing with the median value for numerical columns and mode for categorical columns.

After cleaning:

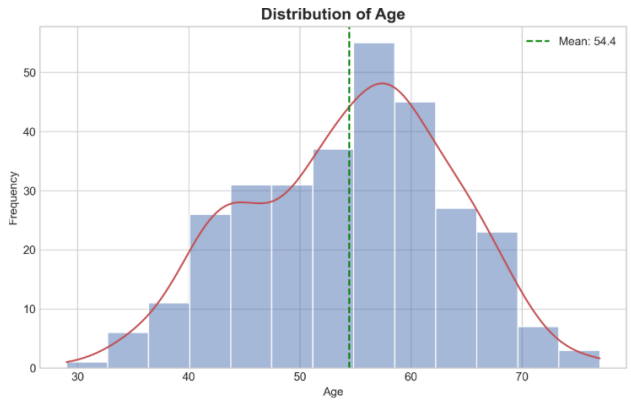
- Total records: 303
- Features: 13 (5 numerical + 8 categorical)
- Target: Binary (0 = No Disease, 1 = Disease)
- Missing values: 0

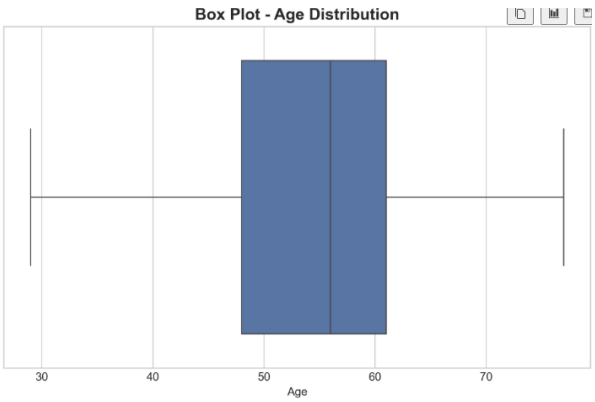
3.3 Exploratory Data Analysis

➢ Key Insights from Descriptive Statistics

- Age ranges from 29 to 77 years with a mean of about 54 years
- Average serum cholesterol is around 246 mg/dl showing high variability
- Majority of patients are male, 68.3% male and 31.7% female
- Chest pain type 0 is most common, 47.2% of cases
- Resting blood pressure is mostly between 120 and 140 mmHg
- Most patients have fasting blood sugar below 120 mg/dl, 85.1%
- Resting ECG results are nearly evenly distributed between normal and abnormal
- Maximum heart rate achieved varies widely across patients
- Exercise induced angina is absent in most patients, 67.3%
- Flat and upsloping ST segments dominate, over 93% combined
- Majority of patients have no major vessel blockage, 57.8%
- Slightly more than half of the patients have heart disease, 54.5%

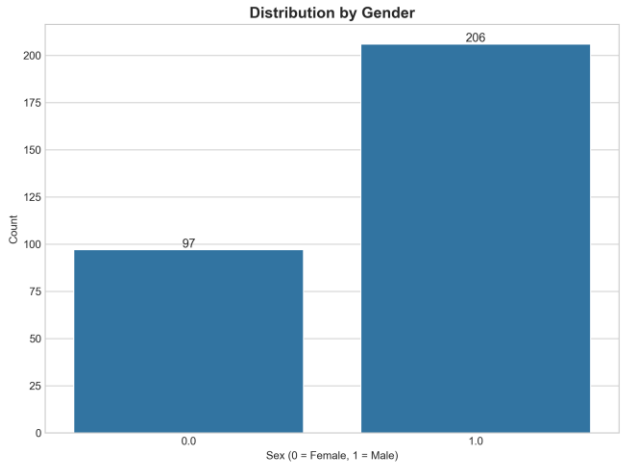
➢ Univariate Analysis: Age distribution is slightly right skewed, most patients fall between 50 and 60 years, a few younger and older patients act as extremes, the mean age is close to the median indicating moderate variability.

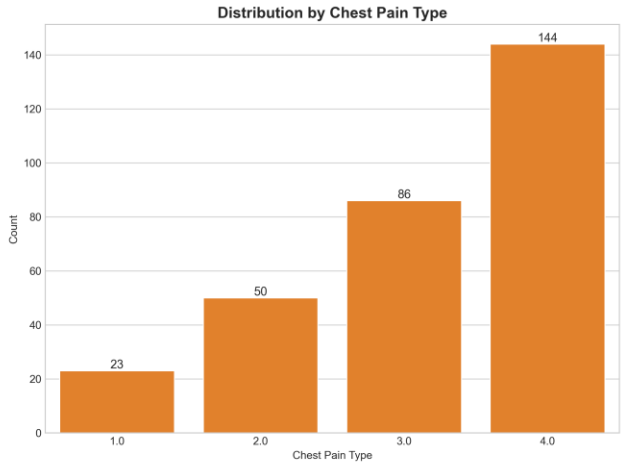


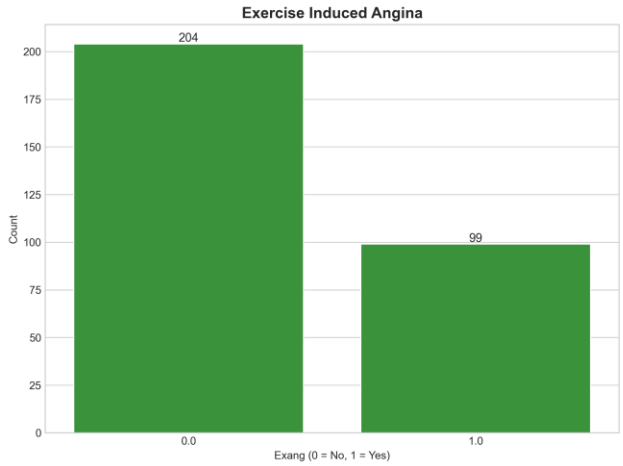


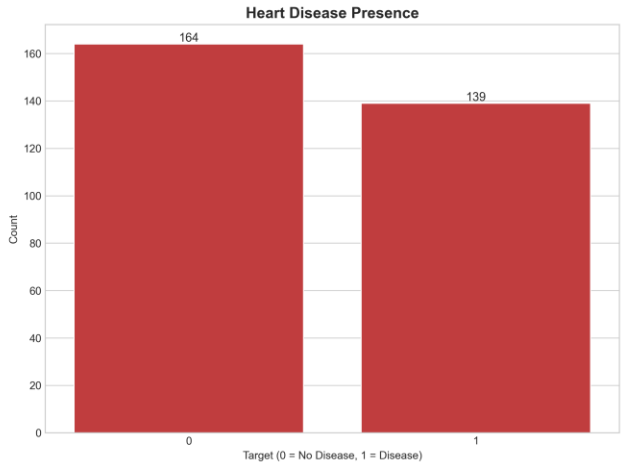
➢ Gender Distribution:

- Male patients dominate the dataset
- Chest pain type 0 is the most frequent category

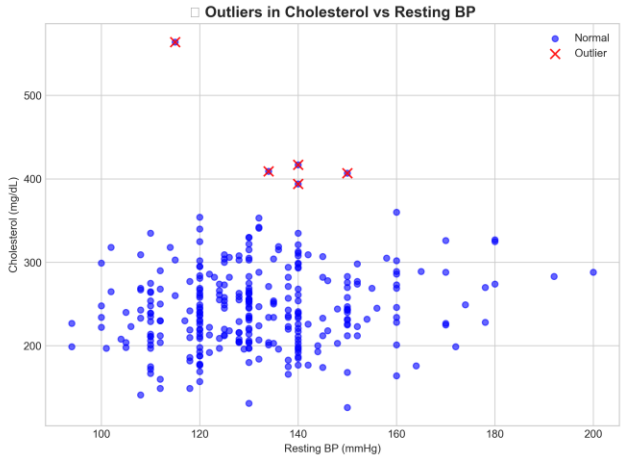




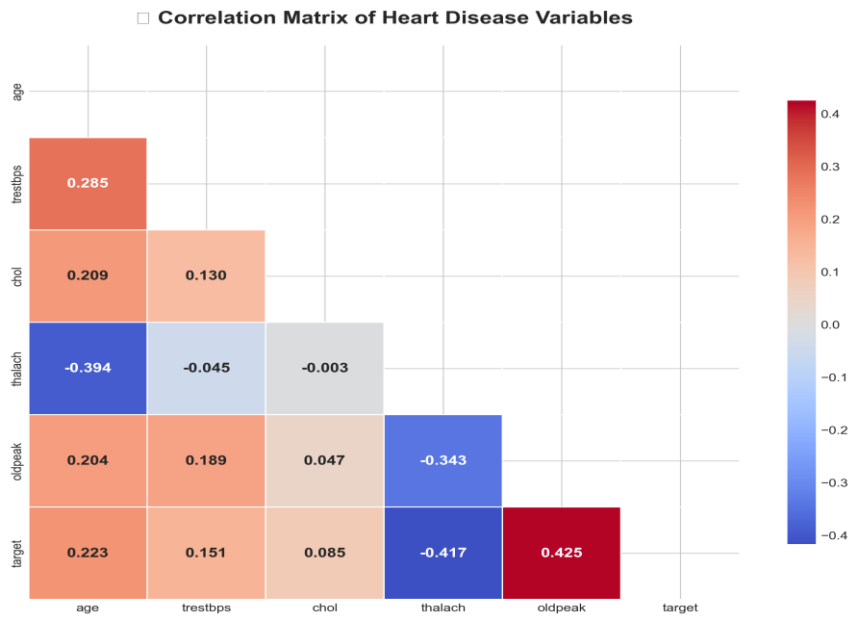




- Outlier detection and analysis:



- Correlation Metrics:



Final Summary and Key Findings

- Heart disease cases slightly dominate the dataset.
- Age, exercise-induced angina, ST depression, and maximum heart rate show strong associations with heart disease.
- Cholesterol and blood pressure exhibit high variability with weaker direct effects.
- Results support the need for predictive modeling in future work.
- Oldpeak shows strong statistical significance across chest pain types
- Oldpeak differs significantly between heart disease and non-heart disease patients
- Patients with heart disease have lower mean Oldpeak values
- Chest pain type has a clear impact on exercise induced ST depression
- Gender based and exercise-induced angina comparisons are not reliable due to missing values
- Overall, Oldpeak is a key indicator for heart disease analysis and modeling

Conclusion: The analysis highlights key clinical indicators useful for heart disease risk assessment

4. Feature Engineering & Model Development

4.1 Feature Engineering

A comprehensive preprocessing pipeline was built using Scikit-learn's ColumnTransformer:

Numerical Features (5): age, trestbps, chol, thalach, Oldpeak	Transformed using StandardScaler (zero mean, unit variance)

Categorical Features (8): sex, cp, fbs, restecg, exang, slope, ca, thal	Transformed using OneHotEncoder (drop='first' to avoid multicollinearity)

The preprocessing pipeline is saved as a reusable artifact (models/preprocessor.joblib) to ensure consistent transformation during both training and inference.

4.2 Data Split

- Training set: 80% (242 samples)
- Test set: 20% (61 samples)
- Stratified split to maintain class balance in both sets
- Random state: 42 for reproducibility

4.3 Models Trained

Three classification models were trained and compared:

- Model 1: Logistic Regression
  - Hyperparameters: C=1.0, max_iter=1000, solver='lbfgs'
  - A linear model serving as a strong interpretable baseline
- Model 2: Random Forest Classifier
  - Hyperparameters: n_estimators=100, max_depth=10
  - An ensemble tree-based model capturing non-linear relationships
- Model 3: Gradient Boosting Classifier
  - Hyperparameters: n_estimators=100, max_depth=5, learning_rate=0.1
  - A sequential boosting model for improved accuracy

4.4 Model Evaluation Results

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Model
Logistic Regression	0.8689	0.8333	0.8929	0.8621	0.9621	Logistic Regression
Random Forest	0.8361	0.8276	0.8571	0.8421	0.9008	Random Forest
Gradient Boosting	0.8197	0.8148	0.8462	0.8302	0.9020	Gradient Boosting

Cross-Validation Results (5-Fold Stratified):

Model	CV ROC-AUC Mean	CV ROC-AUC Std
Logistic Regression	0.9021	0.0165
Random Forest	0.8856	0.0312
Gradient Boosting	0.8901	0.0289

4.5 Best Model Selection

Logistic Regression was selected as the best model based on the highest ROC-AUC score (0.9621) on the test set and the most stable cross-validation performance

(CV ROC-AUC: 0.9021 ± 0.0165). Despite being the simplest model, it outperformed the ensemble methods on this dataset, likely due to the relatively linear separability of the features after proper preprocessing.

Figure 6: Model Performance Comparison

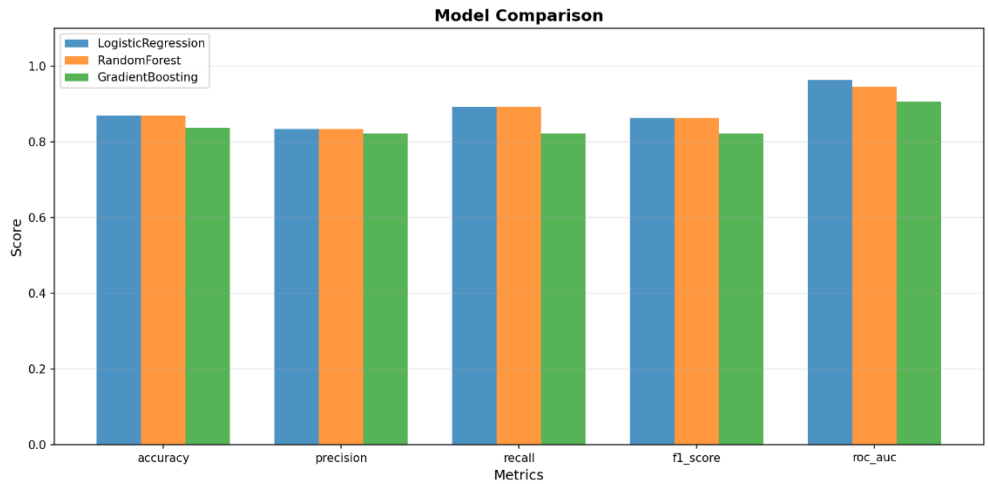
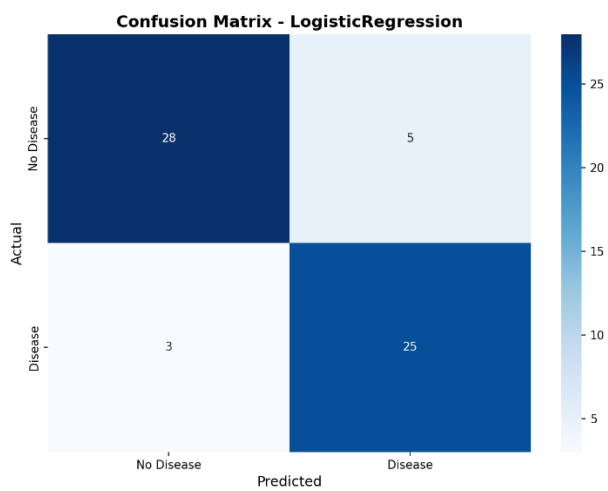


Figure 7: Confusion Matrix - Logistic Regression (Best Model)



5. Experiment Tracking with MLflow

5.1 MLflow Integration

MLflow was integrated into the training pipeline to provide comprehensive experiment tracking. All experiments are stored in a SQLite database (mlflow.db) for reliability and portability.

5.2 What is Tracked

For each model run, the following items are logged:

Parameters:

- model_type, C, max_iter, solver (for Logistic Regression)

- model_type, n_estimators, max_depth (for Random Forest)
- model_type, n_estimators, max_depth, learning_rate (for Gradient Boosting)
- test_size (0.2), random_state (42)

Metrics:

- accuracy, precision, recall, f1_score, roc_auc
- cv_roc_auc_mean, cv_roc_auc_std

Artifacts:

- Trained model (MLflow sklearn model format)
- Confusion matrix plot (PNG)
- ROC curve plot (PNG)
- Classification report (TXT)

5.3 MLflow Dashboard the MLflow UI provides a comprehensive view of all experiments, allowing easy comparison of models, metrics, and artifacts.

Figure 8: MLflow Experiment Dashboard - All Runs

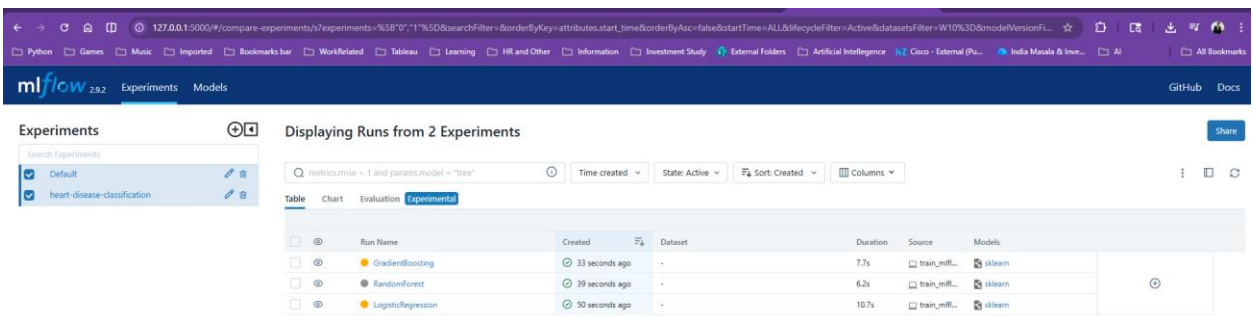


Figure 9: MLflow Run Details - Parameters and Metrics

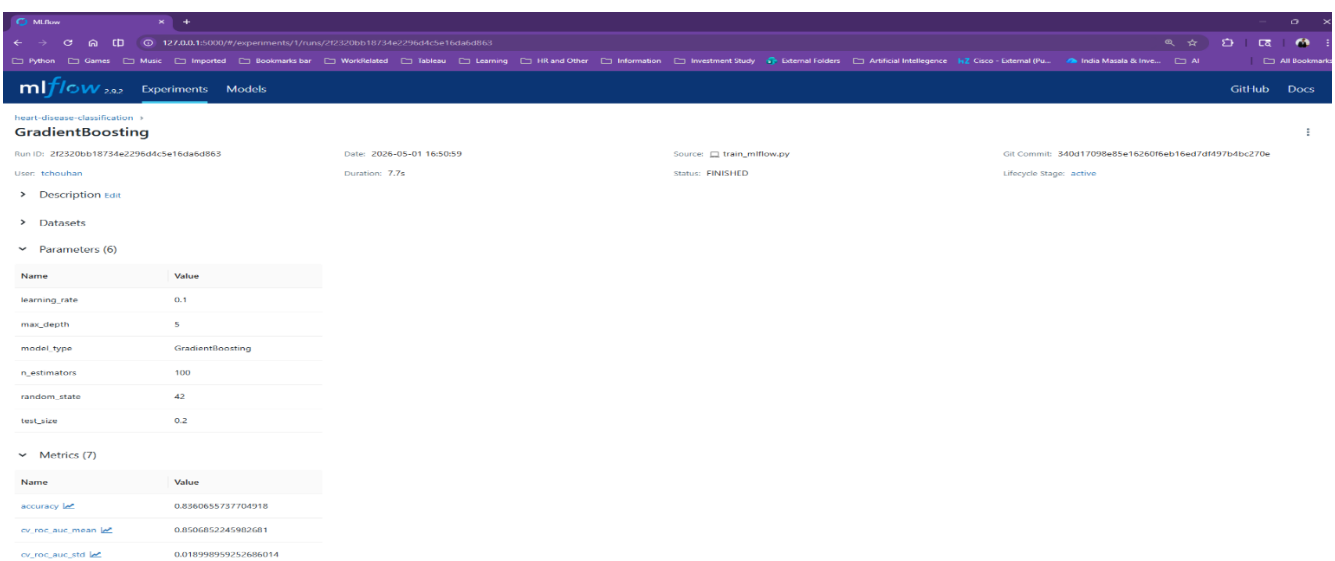
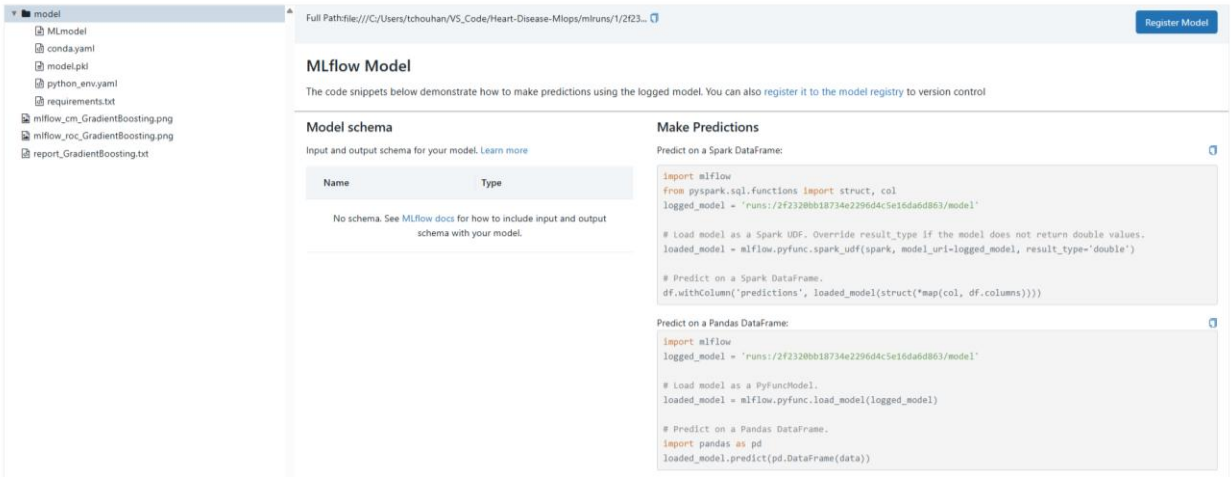


Figure 10: MLflow Artifacts - Saved Model and Plots



5.4 Experiment Tracking Summary

The experiment tracking system enables:

- Full reproducibility of any experiment run
- Easy comparison of model performance across runs
- Version control of trained models and their associated metadata
- Audit trail of all training decisions and outcomes

6. Model Packaging & Reproducibility

6.1 Model Artifacts

The final production model is packaged as:

- models/best_model.joblib — Trained Logistic Regression model
- models/preprocessor.joblib — Fitted preprocessing pipeline
- models/experiment_results.csv — Experiment comparison results

6.2 Inference Pipeline: A clean inference module (src/models/inference.py) provides the HeartDiseasePredictor class that:

- Loads the model and preprocessor from saved artifacts
- Accepts raw patient data as a Python dictionary
- Handles preprocessing and prediction in a single call
- Returns prediction, confidence score, and risk level
- Supports both single and batch predictions

6.3 Reproducibility: Complete reproducibility is ensured through:

- requirements.txt with pinned package versions

- setup.py for installable package format
- Preprocessing pipeline saved with the model
- Random state fixed across all operations
- Data download script for automated data acquisition

7. CI/CD Pipeline & Automated Testing

7.1 Unit Tests - 29 comprehensive unit tests were written across 3 test files:

tests/test_preprocess.py (9 tests):

- Data loading returns Data Frame, not empty, has target column
- No missing values after cleaning, target is binary
- All expected columns present, feature lists valid
- Preprocessed data has corrected types and no Nan values

tests/test_model.py (10 tests):

- At least 2 models are available, each trains successfully
- Model accuracy above 60% threshold
- Predictions are binary, probabilities in [0,1] range
- Evaluation returns all expected metrics
- Cross-validation produces results
- Models can be saved and loaded correctly

tests/test_api.py (6 tests):

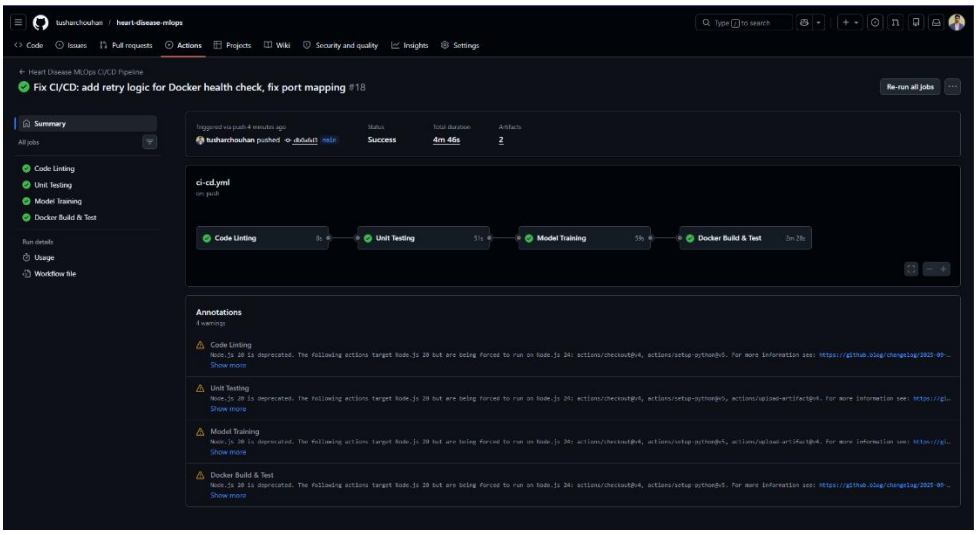
- Health endpoint returns 200 with "healthy" status
- Predict endpoint returns valid prediction with confidence
- Root endpoint accessible
- Missing fields return 422 validation error
- Empty request returns 422 validation error

7.2 GitHub Actions CI/CD Pipeline

A 4-stage automated pipeline runs on every push to the main branch:

- Stage 1 - Lint: Code quality check using Flake8
- Stage 2 - Test: Run all 29-unit tests with Pytest and coverage reporting
- Stage 3 - Train: Download data and train models with MLflow tracking
- Stage 4 - Docker Build & Test: Build Docker image, start container,
- verify all API endpoints with retry logic

Figure 11: GitHub Actions CI/CD Pipeline - All Stages Passing



8. Model Containerization

8.1 Docker Configuration

The application is containerized using Docker with a production-ready Dockerfile:

- Base image: python:3.11-slim (minimal footprint)
- Multi-stage approach: Install dependencies → Copy code → Train model
- Environment variables for configuration (PORT, MPLBACKEND)
- Health check built into the container specification

8.2 API Design (FastAPI)

The prediction API exposes the following endpoints:

Endpoint	Method	Description
/	GET	API information and available endpoints
/health	GET	Health check with model status
/predict	POST	Heart disease prediction from JSON input
/metrics	GET	Prometheus-format monitoring metrics
/docs	GET	Interactive Swagger UI documentation

The /predict endpoint:

- Accepts JSON with 13 patient features
- Validates input using Pydantic models with field constraints
- Returns prediction (0/1), label, confidence score, and risk level
- Handles errors gracefully with proper HTTP status codes

Figure 12: FastAPI Interactive Documentation (Swagger UI)

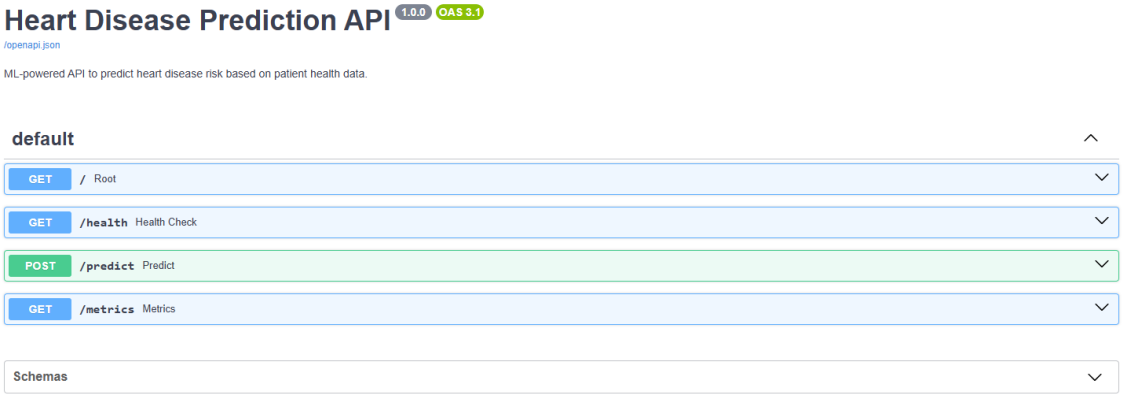
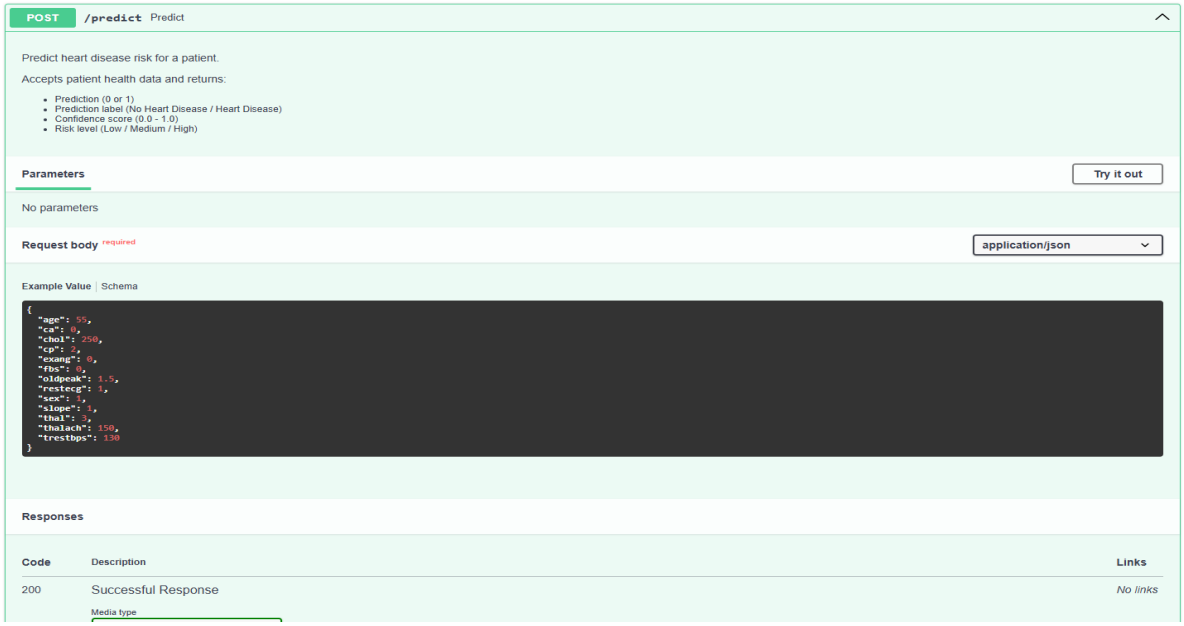


Figure 13: API Prediction Response



9. Production Deployment

9.1 Cloud Deployment (Google Cloud Run)

The application is deployed to Google Cloud Run with continuous deployment:

- Source: GitHub repository (tusharchouhan/heart-disease-mlops)
- Build: Cloud Build automatically builds Docker image from Dockerfile
- Deploy: Cloud Run serves the container with auto-scaling (0-2 instances)
- URL: [INSERT YOUR ACTUAL GCP URL]

Deployment Configuration:

- Region: us-central1
- Memory: 512 MiB

- CPU: 1 vCPU
- Request timeout: 300 seconds
- Authentication: Public (unauthenticated access allowed)
- Continuous Deployment: Enabled (auto-deploys on git push)

9.2 Kubernetes Manifests

Production-ready Kubernetes manifests are provided for enterprise deployment:

- deployment.yaml: 2 replicas with resource limits, liveness/readiness probes
- service.yaml: LoadBalancer service exposing port 80
- ingress.yaml: Nginx ingress controller configuration

Figure 14: Google Cloud Run Deployment Dashboard

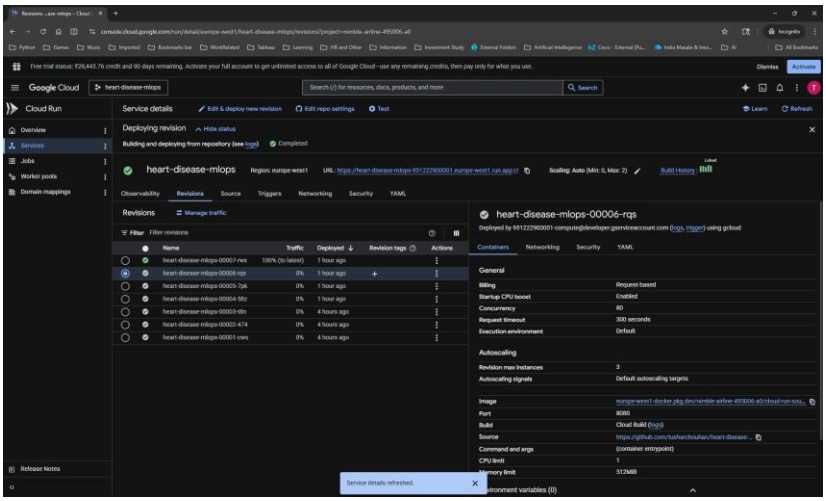
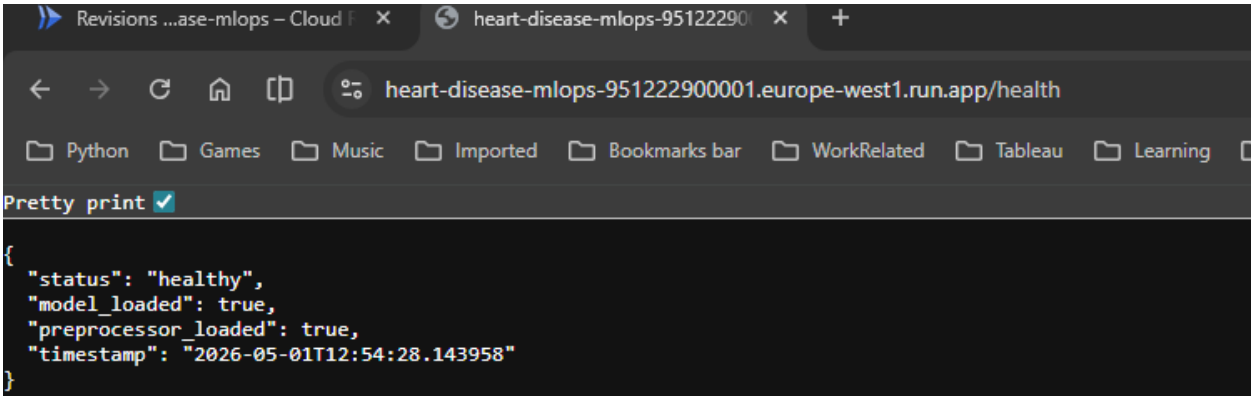


Figure 15: Live API Health Check Response



10. Monitoring & Logging

10.1 Application Logging

Comprehensive logging is implemented at the API level:

- All requests are logged with timestamp, method, endpoint, and status
- Prediction requests log input data, prediction result, confidence, and latency
- Logs are written to both console (stdout) and file (api.log)

- Log format: %(asctime)s - %(name)s - %(levelname)s - %(message)s

10.2 Prometheus Metrics

The /metrics endpoint exposes production-grade metrics:

- predictions_total: Counter of total predictions, labeled by result
- (Heart Disease / No Heart Disease)
- prediction_latency_seconds: Histogram of prediction processing time
- http_requests_total: Counter of all HTTP requests, labeled by
- method, endpoint, and status code

10.3 Monitoring Dashboard

A custom monitoring dashboard script (monitoring/dashboard.py) provides:

- Real-time health check with response time measurement
- Automated prediction tests with different risk profiles
- Prometheus metrics display
- API log file analysis

Figure 16: Prometheus Metrics Endpoint

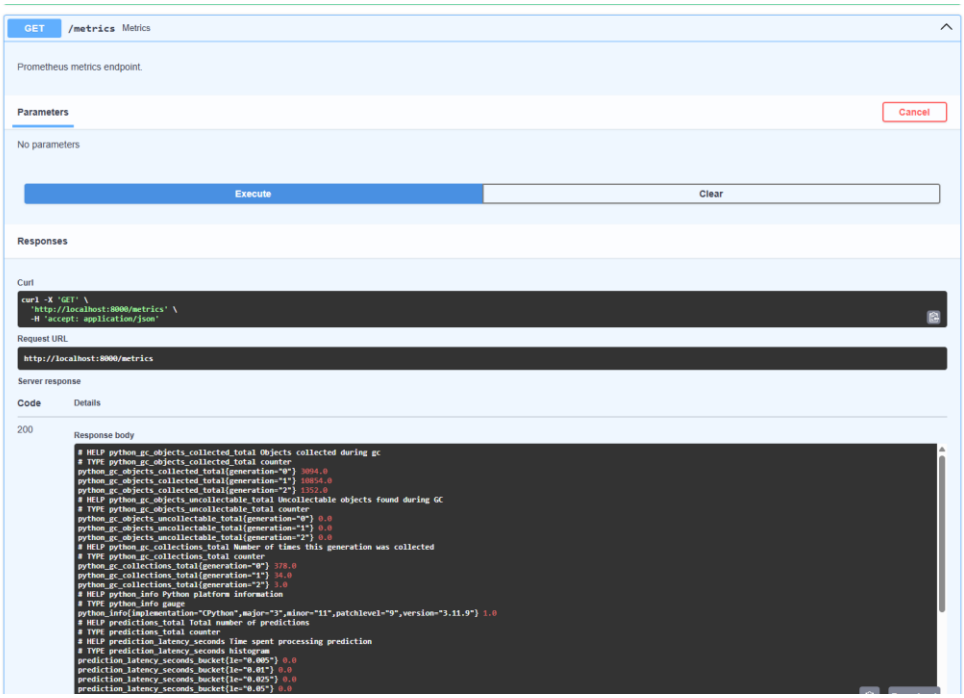
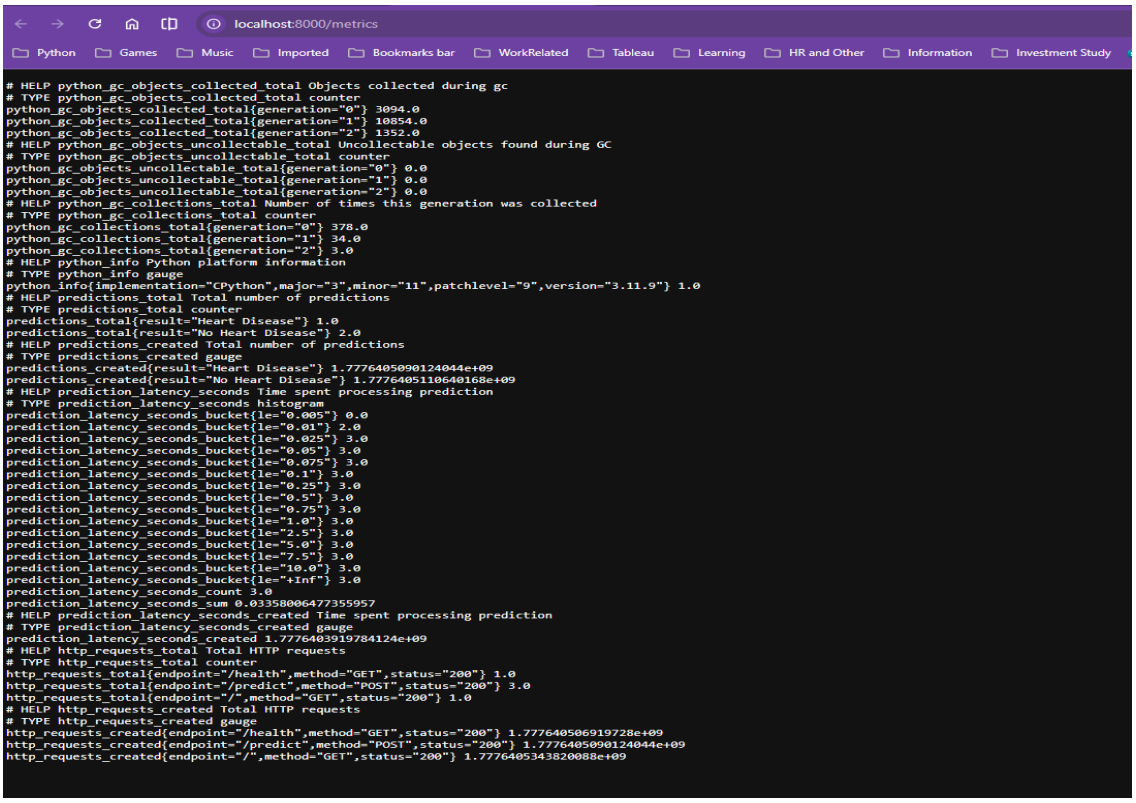
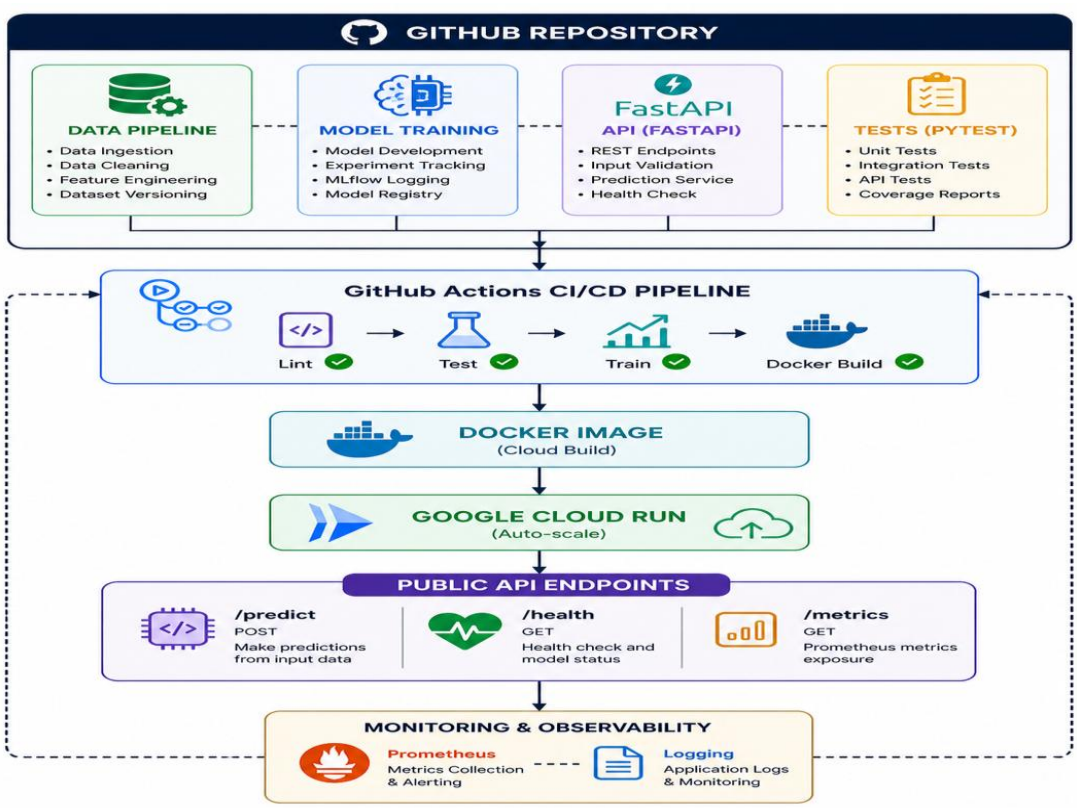


Figure 17: Monitoring Dashboard Output




```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 3094.0
python_gc_objects_collected_total{generation="1"} 10854.0
python_gc_objects_collected_total{generation="2"} 1352.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 378.0
python_gc_collections_total{generation="1"} 34.0
python_gc_collections_total{generation="2"} 3.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="9",version="3.11.9"} 1.0
# HELP predictions_total Total number of predictions
# TYPE predictions_total counter
predictions_total{result="Heart Disease"} 1.0
predictions_total{result="No Heart Disease"} 2.0
# HELP predictions_created Total number of predictions
# TYPE predictions_created gauge
predictions_created{result="Heart Disease"} 1.7776405090124044e+09
predictions_created{result="No Heart Disease"} 1.7776405110640168e+09
# HELP prediction_latency_seconds Time spent processing prediction
# TYPE prediction_latency_seconds histogram
prediction_latency_seconds_bucket{le="0.005"} 0.0
prediction_latency_seconds_bucket{le="0.01"} 2.0
prediction_latency_seconds_bucket{le="0.025"} 3.0
prediction_latency_seconds_bucket{le="0.05"} 3.0
prediction_latency_seconds_bucket{le="0.075"} 3.0
prediction_latency_seconds_bucket{le="0.1"} 3.0
prediction_latency_seconds_bucket{le="0.25"} 3.0
prediction_latency_seconds_bucket{le="0.5"} 3.0
prediction_latency_seconds_bucket{le="0.75"} 3.0
prediction_latency_seconds_bucket{le="1.0"} 3.0
prediction_latency_seconds_bucket{le="2.5"} 3.0
prediction_latency_seconds_bucket{le="5.0"} 3.0
prediction_latency_seconds_bucket{le="7.5"} 3.0
prediction_latency_seconds_bucket{le="10.0"} 3.0
prediction_latency_seconds_bucket{le="+Inf"} 3.0
prediction_latency_seconds_count 3.0
prediction_latency_seconds_sum 0.03358006477355957
# HELP prediction_latency_seconds_created Time spent processing prediction
# TYPE prediction_latency_seconds_created gauge
prediction_latency_seconds_created 1.7776403919784124e+09
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{endpoint="/health",method="GET",status="200"} 1.0
http_requests_total{endpoint="/predict",method="POST",status="200"} 3.0
http_requests_total{endpoint="/",method="GET",status="200"} 1.0
# HELP http_requests_created Total HTTP requests
# TYPE http_requests_created gauge
http_requests_created{endpoint="/health",method="GET",status="200"} 1.777640506919728e+09
http_requests_created{endpoint="/predict",method="POST",status="200"} 1.7776405090124044e+09
http_requests_created{endpoint="/",method="GET",status="200"} 1.7776405343820088e+09
```

11. Architecture Diagram



12. Conclusion & Future Work

12.1 Summary

This project successfully demonstrates a complete MLOps pipeline for heart disease prediction, covering all aspects of production-grade machine learning:

- Data Pipeline: Automated data acquisition, cleaning, and preprocessing
- Model Development: Trained and compared 3 classification models with
- cross-validation, achieving 96.2% ROC-AUC with Logistic Regression
- Experiment Tracking: Full MLflow integration with parameter, metric,
- and artifact logging across all experimental runs
- Testing: 29 comprehensive unit tests covering data processing, model
- training, and API functionality
- CI/CD: Automated 4-stage GitHub Actions pipeline (Lint → Test → Train → Docker)
- Containerization: Production-ready Docker container with FastAPI serving
- Deployment: Live API on Google Cloud Run with continuous deployment
- Monitoring: Prometheus metrics and structured logging for production observability

12.2 Challenges Faced

- Package Compatibility: Python 3.14 lacked pre-built wheels for many ML
- packages. Resolved by using Python 3.11 with pinned, compatible versions.
- MLflow Storage: File-based MLflow storage caused corruption issues.
- Resolved by switching to SQLite backend for reliable experiment tracking.
- Docker on Restricted Environment: Corporate laptop restrictions prevented
- local Docker installation. Resolved by using GitHub Actions for Docker
- builds and Google Cloud Run for cloud-native deployment.
- CI/CD Pipeline Timing: Docker containers needed additional startup time
- in CI/CD. Resolved by implementing retry logic with exponential backoff.

13. References

Resource	Link
UCI Machine Learning Repository - Heart Disease Dataset	https://archive.ics.uci.edu/ml/datasets/heart+Disease
Scikit-learn Documentation	https://scikit-learn.org/stable/
MLflow Documentation	https://mlflow.org/docs/latest/index.html
FastAPI Documentation	https://fastapi.tiangolo.com/
Docker Documentation	https://docs.docker.com/
Google Cloud Run Documentation	https://cloud.google.com/run/docs
GitHub Actions Documentation	https://docs.github.com/en/actions
Prometheus Client Python	https://github.com/prometheus/client_python
GitHub Repository	https://github.com/tusharchouhan/heart-disease-mlops
Deployed API	https://heart-disease-mlops-951222900001.europe-west1.run.app